Taking a RISC-V Processor from Verilog to a Finished Chip Layout

A five-stage RV32I core, verified and physically implemented in SkyWater 130 nm with open-source tools

Arjanveer Samra

What this project is, and why I built it

I wanted to find out what actually happens between writing Verilog and having something a foundry could build. In my computer architecture coursework I designed a multi-cycle RISC-V CPU and ran it on an FPGA, which was satisfying, but an FPGA hides the entire back end. You never place a cell, never route a wire, and never find out whether your design would still work when the silicon is slow and hot. I wanted the rest of it.

So I built a five-stage pipelined RV32I processor and took it all the way to a GDSII layout in the SkyWater 130 nm open process, using only free tools. The design is verified three different ways, placed, routed, and checked against the foundry design rules. It passes DRC and LVS, and it meets both setup and hold timing at three process corners.

Two things shape how the rest of this reads. The verification found a bug that no amount of simulation would have caught, and I think that is the most interesting result here. And an earlier version of this design looked finished and was not: it passed everything at the typical corner and would have been a dead chip in real silicon. Finding that out changed what I think "done" means.

The short version

	Result
Frequency guaranteed across process, voltage and temperature	32.3 MHz
Frequency on typical silicon only	69.0 MHz
Setup slack, slow corner	+0.8409 ns
Hold slack, fast corner	+0.0945 ns
Power	3.03 mW
Die size	424.58 × 427.04 µm
Cells placed, including fill	25,218
Routed wire	315,103 µm
Instructions checked against a reference model, zero mismatches	39,244 across 144 programs
Functional coverage	103 of 103 bins
Formal proofs	5 of 5 pass, 2 of them unbounded
Design rule check	0 violations (64 found and fixed)
Layout versus schematic	Netlists match
Electrical checks still failing	129 slew, 8 capacitance
Bugs found and fixed along the way	5

Every number in this report comes out of a tool report file. I wrote the report generator to pull them straight from those files, because before I did that I quoted a stale number more than once without noticing.

1. The tools, and getting them to run

Everything here is open source and runs on a single CPU core with 4 GB of RAM. No commercial EDA software is involved at any point.

Job	Tool	Version
Simulation	Icarus Verilog	12.0
Linting	Verilator	5.020
Testbenches	cocotb	2.1.0
Formal proofs	SymbiYosys with Yosys	0.33
SMT solver	z3	5.1.0
Synthesis	Yosys and ABC	0.33
Place and route, timing	OpenROAD	PyPI wheel
Parasitic extraction	OpenRCX	bundled with OpenROAD
Layout, DRC, LVS	KLayout	0.28.16
Process design kit	SkyWater sky130A, high-density library	ciel ef60e1e8
Platform files and rule decks	OpenROAD-flow-scripts sky130hd	commit 95ebc50
Compiler	riscv64-unknown-elf-gcc	13.2.0
Compliance tests	riscv-software-src/riscv-tests	commit 87b04f0

Table 1. Tools and versions.

Getting OpenROAD running took longer than I expected, and the workarounds are worth writing down because none of them are in any tutorial.

- There is no OpenROAD binary for this machine. Not in the package manager, not in the project's own releases. The only thing on PyPI is a wheel that ships an embedded Tcl interpreter behind a Python API, with no command-line tool at all. I wrote a shim that translates the normal command line onto that API so standard scripts run unchanged.
- The floorplan commands crash. Calling `initialize_floorplan` through the Tcl interpreter segfaults every time. The same operation through the Python `InitFloorplan` class works, so the flow builds the die and core rectangles in Python and does everything else in Tcl.
- The z3 that ships with Ubuntu, version 4.8.12, dies partway through the ALU and decoder proofs with "unexpected EOF response from solver". Installing 5.1.0 from PyPI fixed it and both proofs then finished in under a second.

2. How the processor works

The core is a five-stage in-order pipeline: fetch, decode and register read, execute, memory, write-back. It has separate instruction and data memories, both of which return data in the same cycle the address is presented. I kept it deliberately small: no cache, no multiply or divide, no CSRs, no interrupts. The point was to get the base integer instruction set and the hazard logic exactly right, not to build something impressive on paper.

2.1 Hazards

Three things can go wrong in a pipeline, and the core handles each in hardware.

- **An instruction needs a result that has not been written back yet.** Results are forwarded into execute from two places, the EX/MEM and MEM/WB pipeline registers. The third case, where the producer is in write-back and the consumer is in decode, is handled inside the register file: a read port returns the value being written in the same cycle if the addresses match. That saved a whole forwarding path.

- **A load feeds the very next instruction.** The data is not back in time, so the pipeline stalls one cycle and pushes a bubble into execute.
- **A branch goes the way we did not guess.** Branches resolve in execute and the core assumes not taken, so a taken branch throws away two cycles of work. Nothing is written speculatively, so a flushed instruction leaves no trace.

Across the whole regression this works out to 0.869 instructions per cycle, or 1.151 cycles per instruction, over 39,244 retired instructions. That is about what you would expect from a machine that issues one instruction per cycle except when it stalls for a load or flushes for a branch.

2.2 What happens on an instruction the core cannot run

There is no trap handler, so anything the core cannot execute retires with a trap flag, does nothing to registers or memory, leaves the program counter where it was, and halts the front end. That covers ECALL, EBREAK, every CSR instruction, all illegal encodings, and later on a jump to a misaligned address. Handling them all the same way is what let me write a reference model that could check the behaviour exactly.

2.3 The files

File	What it does
rv32i_core.v	The pipeline: hazards, forwarding, memory interface
rv32i_decode.v	Instruction decoder, raises a trap on anything illegal
rv32i_alu.v	The ten integer operations
rv32i_regfile.v	32 registers, two read ports, writes visible in the same cycle
tb_top.v	Simulation wrapper with 16 KiB of instruction and data memory

Table 2. The four synthesizable files plus the testbench wrapper. The instruction trace port used for verification only exists when a macro is defined, so it is not in the chip.

3. Verification

This is the part I spent the most time on. The reason it is layered is that each technique catches a kind of bug the others miss. Directed tests catch the corner cases you can think of. Random tests catch interactions you cannot. Coverage tells you whether the random tests actually went where you hoped. And formal proves things about every possible input, including inputs no simulation will ever generate. Section 6.1 is the bug that justifies that last layer.

3.1 A reference model I wrote separately

I wrote a RISC-V simulator in Python straight from the specification, deliberately without looking at my own RTL while doing it. If you write both from the same mental picture, a misunderstanding lives in both and the comparison passes anyway. Before trusting it I ran the official `rv32ui` compliance suite on it: all 40 self-checking programs report their pass code.

3.2 Comparing the core against it, instruction by instruction

The core exports a trace of every instruction it retires. The testbench steps the reference model once for each retired instruction and compares all sixteen fields: the instruction word, the trap flag, the program counter before and after, both source registers and their values, the destination register and its value, and the memory address, masks and data.

Comparing per retired instruction rather than per cycle matters. Stalls and flushes do not confuse the checker, but an instruction that retires when it should have been thrown away gets caught immediately, because the reference model never produces it.

Suite	Programs	Instructions	Cycles	IPC
Official RISC-V compliance tests	40	11,828	13,152	0.899
Directed corner cases	56	14,911	16,605	0.898
Constrained random	48	12,505	15,417	0.811
Total	144	39,244	45,174	0.869

Table 3. Every program matched the reference model on every field of every instruction.

3.3 The directed tests

I generated these programmatically instead of hand-writing them, so I could sweep whole matrices rather than only the cases I happened to think of:

- All 13 corner values (zero, one, minus one, the signed and unsigned extremes, alternating bit patterns, shift boundaries) crossed against each other for all ten register-register operations, including the cases where the destination is also a source.
- Every immediate and every interesting shift amount, including shift amounts above 31 that have to be truncated to five bits.
- A hazard matrix: seven kinds of producer, nine consumer roles, at distances of one to four instructions, with and without an older write to the same register that has to be overridden. Branch cases are emitted twice so both outcomes happen at each distance.
- Byte and half-word accesses at every lane of a word, signed and unsigned, plus store-to-load distances and offsets at the ends of the immediate range.
- Writes to x0 from every instruction class at every hazard distance, checking the write is discarded and reads still return zero.

- 26 illegal encodings, each checked to trap with nothing written.
- Illegal instructions placed on the not-taken side of a branch and after a jump, which must never trap because they are never really executed.

3.4 The random tests

Four stimulus mixes: balanced, hazard-heavy, memory-heavy and branch-heavy. The generator guarantees every program is legal and terminates, by building addresses from a fixed base register so they always land inside the data window, and computing jump targets relative to the program counter so control flow cannot escape.

3.5 Coverage, and why the last two bins were the interesting ones

Running a lot of instructions is not the same as testing a lot of behaviour, so I built a coverage model of 103 bins in 12 groups: instruction mix, result classes, shift ranges, branch directions and distances, memory lanes, sign extension, x0 handling, operand corners, which forwarding path was actually used, and control transfer types.

Group	Bins	Hit	What it measures
insn	39	39	every instruction, plus illegal
alu_result	6	6	result classes including unsigned wrap-around
shift_amount	6	6	shift amounts, including 0 and 31
branch_dir	12	12	each branch type, taken and not taken
branch_offset	3	3	backward, short forward, long forward
mem_lane	7	7	byte, half and word accesses at each lane
load_sign	3	3	sign extension versus zero extension
rd_zero	2	2	writes to x0 versus a real register
src_zero	3	3	x0 read as a source
operand_corner	6	6	corner operand values
hazard	9	9	which forwarding path or stall actually happened
control	7	7	transfer types, including back-to-back and traps
Total	103	103	

Table 4. The hazard group is measured from the instruction trace itself, so a bin only counts when a producer and consumer really did land at that distance.

Closing the last two bins took work rather than more random seeds, which is the whole reason to measure coverage. One was long forward branches, which the generators structurally could not produce because they place branch targets close by, so I wrote a directed program with branches reaching 200 instructions ahead. The other turned out to be a bug in my coverage model: I had declared a bin for unsigned wrap-around of the adder and never written the code that credits it. A bin that cannot be hit is worse than no bin at all, because it quietly drags your coverage number down and sends you hunting for stimulus that already exists.

3.6 Formal proofs

Simulation shows the cases you ran are right. Formal shows a property holds for every input, including ones you would never think to generate. I wrote five property suites as independent restatements of what the specification requires, not as copies of my own logic. All five pass. Two are proved by k-induction, which means no counterexample exists at any depth at all, not just within some window I picked.

Suite	Method	Depth	Some of the properties
ALU	combinational	1	subtraction undoes addition; comparison is antisymmetric; signed and unsigned comparison agree exactly when the sign bits agree; arithmetic right shift keeps the sign
Decoder	combinational	1	legality matches an opcode table I wrote separately from the spec; trapping instructions have no side effects; every immediate matches its format
Register file	k-induction	unbounded	x0 always reads zero; a write is visible to a read in the same cycle; the two read ports agree
Register file retention	bounded	15	a written value comes back until something overwrites it
Core invariants	k-induction	unbounded	fetches are always word aligned; x0 is never written; memory masks are contiguous and aligned; a trap does not move the program counter; halt is sticky

Table 5. In the core proofs the instruction and data memories are left completely free, so the proof covers every program that could exist, not the ones in my regression.

Two things I learned writing these. The decoder proof only means something because I wrote the legality table in the property file from the specification, separately from the decoder; proving a design equals itself proves nothing. And the register retention property is not inductive on its own, because k-induction can start from a state where my shadow copy and the real register disagree. Rather than weaken the property so it would pass, I check retention with bounded model checking to depth 15, prove the rest unbounded, and say which is which.

3.7 Checking the netlist, not just the RTL

Verifying the RTL does not verify what synthesis actually produced. So I simulated the synthesized netlist against the SkyWater cell models with 45 programs. The netlist has no trace port, since that logic only exists in simulation builds, so instead of comparing instruction by instruction I compared the end state: all 4,096 words of data memory have to match the reference model when the program finishes. All 45 pass, and they run cycle for cycle identically to the RTL.

4. Synthesis

Yosys maps the design onto the SkyWater high-density standard cells, with ABC doing technology mapping in timing-driven mode. The 31-entry register file becomes flip-flops, because there is no register-file compiler in this PDK flow, and that is what dominates the area.

	Value
Standard cells	5,811
Flip-flops	1,375
Distinct cell types used	64
Nets	5,719
Cell area before layout	69,077 µm²

Table 6. The flip-flop count lines up with 992 bits of architectural register file plus the pipeline registers.

Cell	Count	Cell	Count
dfxtp_1	1375	mux2_1	1072
mux4_2	577	nand2_1	413
nor2_1	413	a21oi_1	379
o21ai_0	349	mux2i_1	249
xnor2_1	124	xor2_1	106
a22oi_1	54	nand3_1	50
clkinv_1	49	lpflow_isobufsrc_1	45

Table 7. The cells that get used the most.

5. Turning the netlist into a layout

The physical flow is one script that runs every stage in OpenROAD and writes out metrics after each one. Recording metrics per stage is what made the flow debuggable: when something is wrong, the stage where the numbers stop making sense tells you where to look. Section 6.2 is exactly that.

5.1 Settings

Setting	Value
Standard cell library	sky130_fd_sc_hd, high density
Site	unithd, 0.46 × 2.72 µm
Target utilization	40%
Aspect ratio	1.0
Core to die margin	4.6 µm
Routing layers	met1 to met5, clock on met3 to met5
Pin layers	met3 horizontal, met2 vertical
Power grid	met1 follow-pins with met4 and met5 straps
Tap cell spacing	14 µm
Placement density	0.60

Setting	Value
Clock buffers	clkbuf_4, 8 and 16, root clkbuf_16
Clock uncertainty	0.25 ns
I/O timing budget	20% of the clock period at each end

Table 8. Flow configuration.

5.2 Stage by stage

Stage	Instances	Cell area (µm²)	Util. (%)	Setup slack (ns)	Power (mW)
Floorplan	5,811	69,077	40.0	-	-
Timing before placement	-	-	-	5.702	3.35
Tap cells	8,136	71,986	41.6	-	-
Global placement	8,136	71,986	41.6	-4.513	3.68
Repair and detailed placement	8,871	77,224	44.7	5.928	3.31
Clock tree	9,109	81,794	47.3	5.944	6.06
Post-clock-tree optimization	9,109	81,794	47.3	5.944	6.06
Global route	-	-	-	5.567	6.19
Detailed route	9,109	81,794	47.3	-	-
Filler cells	24,891	172,864	100.0	-	-
After extraction	-	-	-	6.047	5.94

Table 9. This walks through the flow on one run at a 20 ns clock. It is not the design that was finally signed off, and every number in it belongs to that 20 ns run. The final results are in Sections 5.4 and 5.5.

Two movements here are worth reading. Global placement shows negative slack because at that point the design still has unbuffered high-fanout nets, and the repair step right after inserts buffers and gets the slack back. Clock tree synthesis adds 132 buffers and roughly doubles the power, which is expected: before that the clock is one ideal net, and afterwards it is a real tree switching every cycle.

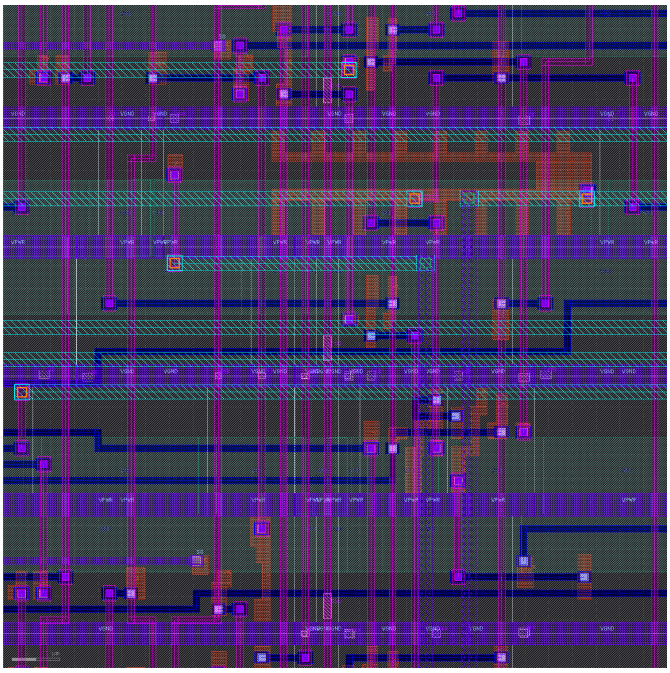


Figure 1. A 14 × 14 µm piece of the finished layout. The horizontal bands are rows of standard cells, each bounded by its power and ground rails. Orange is diffusion, the purple squares are via stacks, and the wiring runs on met1 (dark blue), met2 (magenta, vertical) and met3 (cyan, horizontal), each following its preferred direction.

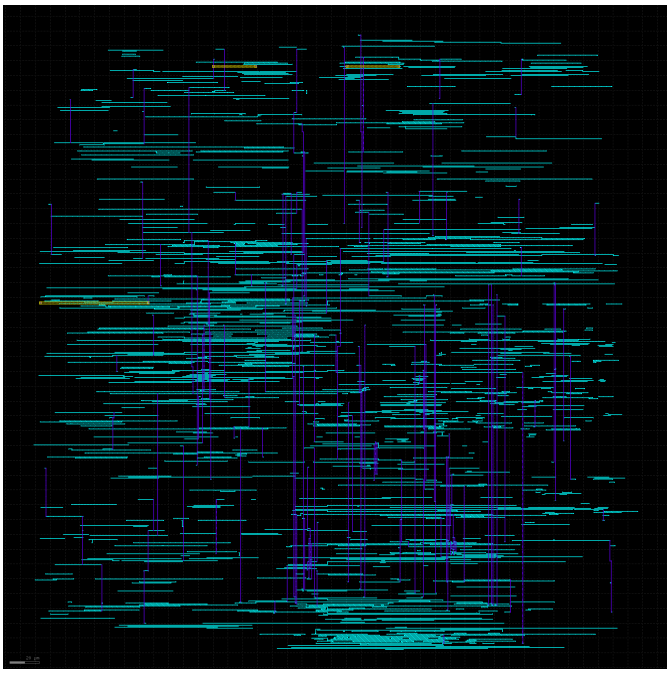


Figure 2. The whole die with only met3 (cyan, horizontal) and met4 (purple, vertical) drawn. I am not showing a view with every layer on, because at this size each 0.3 µm wire is smaller than one pixel and the picture turns into moiré that tells you nothing. With the lower layers hidden you can see the routing get denser through the middle of the core, where the register file and the forwarding multiplexers are.

5.3 How fast does it go

I swept the target clock period to find where the design stops closing. Detailed routing takes about twenty minutes a point on one core and barely moves the slack, so the sweep runs through global route, and then I confirmed the closure point

with a full routed and extracted run.

Period (ns)	Frequency (MHz)	Setup slack (ns)	Power (mW)	
10.00	100.0	-1.5971	12.39	fails
11.00	90.9	-0.9031	11.26	fails
12.00	83.3	-0.8018	10.32	fails
13.00	76.9	-0.5458	9.52	fails
14.00	71.4	-0.3474	8.84	fails
14.50	69.0	+0.0666	8.54	closes
15.00	66.7	+0.5666	8.25	closes
16.00	62.5	+1.5666	7.74	closes
20.00	50.0	+5.5666	6.19	closes

Table 10. The design closes at 14.5 ns and misses at 14.0 ns by 0.35 ns.

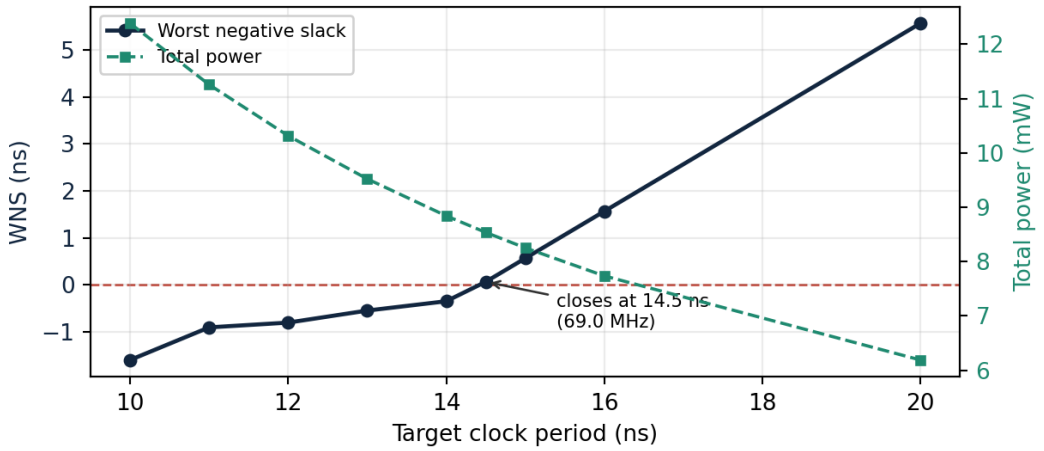


Figure 3. Slack and power against the target clock period. Slack crosses zero between 14.0 and 14.5 ns, and power climbs steadily as the target tightens and the optimizer works harder.

So 14.5 ns, or 69 MHz. Power rises the whole way down, from 6.19 mW at 20 ns to 12.39 mW at 10 ns, because the optimizer keeps upsizing cells and adding buffers to chase timing it cannot actually meet. The failing points are informative too: the total negative slack does not grow smoothly, which tells me the optimizer is making structurally different choices at each target rather than the design having one stubborn path.

	20 ns target	14.5 ns target
Setup slack	+6.0469 ns	+0.5469 ns
Hold slack	+0.0779 ns	+0.0779 ns
Power	5.940 mW	8.194 mW
Routed wire	305,526 µm	305,526 µm
Cells with fill	24,891	24,891

Table 11. Two analyses of one implementation, not two implementations.

One honest note about that table. These are not two separate builds. The DEF and the netlist from the 20 ns and 14.5 ns runs are byte for byte identical, and only the timing constraint differs, which is why the wirelength and cell count match exactly. The 20 ns build already met a 14.5 ns clock without any re-optimization. That is a better result than two builds

would have been, since the speed came out of the layout I already had rather than out of extra buffering, but calling them two implementations would have been wrong.

5.4 Three corners, and the thing that nearly got past me

Everything up to here uses the typical library: nominal process, 25 °C, 1.80 V. That is how the design was optimized and, on its own, it is not a signoff. Real silicon has to work at the slow corner, where cells are slower and setup paths are hardest, and at the fast corner, where cells are quicker and hold paths are at risk. When I re-ran timing on the finished 14.5 ns layout against the slow and fast libraries, this came out:

Corner	Library	Setup slack (ns)	Hold slack (ns)	
ss	slow: 100 C, 1.60 V	-15.3770	+0.4174	setup fails
tt	typical: 25 C, 1.80 V	+0.5469	+0.0779	passes
ff	fast: -40 C, 1.95 V	+6.1207	-0.0471	hold fails

Table 12. The 69 MHz design, checked at all three corners. It works at typical and fails at both ends.

Setup misses by 15.38 ns at the slow corner. That is not a near miss: at 1.60 V and 100 °C the critical path takes roughly twice as long, so the design would need a clock near 30 ns, not 14.5. And hold misses by 47 ps at the fast corner. The hold failure is the one that actually worries me, because you cannot fix hold by slowing the clock down. The chip is just broken. If I had signed this off at the typical corner and stopped, both problems would have gone to a foundry.

My first attempt at a fix was to swap the slow library in and rebuild. Setup closed, and hold still failed at the fast corner by 50 ps. That took me a while to understand: hold repair was looking at the slow corner, where hold already passed by 410 ps, so it saw nothing to fix. You have to analyze the corner that fails, not reason about which one probably matters.

So I rebuilt the flow to be genuinely multi-corner. The corners get declared before any library is loaded, both the slow and fast libraries go in, and setup and hold repair optimize against whichever is worse. That also exposed a gap in my own flow: slack that was positive after the clock tree went negative after global route, because the routed wiring is more pessimistic than the placement estimate and I had no optimization step between the two. Adding a post-route repair pass, which repairs setup and hold against real routed parasitics, legalizes, then re-routes, recovered it.

Two slack numbers show up for the final design and both are real. The flow's own signoff run reports +0.8409 ns at the slow corner, and the standalone corner script reports +0.9422 ns. They use different reporting paths, and the standalone script tolerates a failed parasitics load rather than stopping, so I quote the flow's number throughout this report because that run definitely had the extracted parasitics in it.

	Typical-only build (14.5 ns)	Multi-corner build (31 ns)
Setup slack, slow corner	-15.3770 ns	+0.9422 ns
Hold slack, fast corner	-0.0471 ns	+0.0945 ns
Cells with fill	24,891	25,218
Router DRC	0	0
Signoff DRC	0	0
LVS	match	match
Frequency guaranteed across PVT	does not close	32.3 MHz
Frequency on typical silicon	69.0 MHz	71.2 MHz

Table 13. The two builds. The fast one only works on typical silicon. The slower one works everywhere.

So the honest answer to "how fast is it" is two numbers, not one. **69 MHz on typical silicon, 32.3 MHz guaranteed across process, voltage and temperature.** A factor of about two between them is normal for this process, where the slow

corner drops the supply from 1.80 V to 1.60 V and heats the die to 100 °C. If I only quoted the bigger number I would be describing a chip that does not work.

5.5 Wiring and power for the final design

	Value
Routed wire	315,103 µm
Nets	8,059
Router DRC violations	0
Detailed routing passes to clean	7 (4314 → 2323 → 2029 → 321 → 29 → 10 → 0)
Filler cells	14,796
Internal power	2.192 mW
Switching power	0.784 mW
Leakage	58.45 µW
Total power	3.034 mW
Slew violations still failing	129, worst -0.10 ns
Capacitance violations still failing	8

Table 14. Leakage is tiny next to dynamic power, which is what you expect at this node and this activity level. The slew and capacitance violations are real and unfixed; see Section 8.3.

6. The five bugs

These are in order of how much they matter. What interests me is not that bugs existed but which technique was capable of finding each one.

6.1 A jump to an address that does not exist

What happened. The core-level property that every instruction fetch is word aligned failed at step 4 of the proof, with a counterexample where a branch target came out 2 past a word boundary.

Why. Branch and jump offsets in RISC-V are encoded as multiples of two, not four, because the compressed instruction extension allows instructions on half-word boundaries. My core has no compressed extension, but it was adding the offset to the program counter and redirecting fetch without checking, so a target 2 past a word boundary would have made the front end fetch from an address that does not hold a real instruction.

The fix. Execute now catches a misaligned target, cancels the transfer, and retires the instruction as a trap with nothing written, which is what the specification calls an instruction-address-misaligned exception, expressed the way this core handles every other exceptional case. I updated the reference model to match and added four directed tests.

Why simulation was never going to find it. Every program in my regression came out of an assembler or one of my generators, and both only ever emit targets on word boundaries. More random seeds would not have helped, because the generator was structurally incapable of producing the case. Formal found it in seconds precisely because it does not use a generator: the instruction memory is a free variable, so the solver is free to pick the one encoding that breaks the property. This is the result I am most pleased with, and the clearest argument I have for bothering with formal at all.

6.2 A step that failed silently and looked like it worked

What happened. My first apparently successful place-and-route run reported timing after placement that was bit for bit identical to the timing before placement. That is physically impossible, and it is the only reason I looked.

Why. The `-random` option of `place_pins` is obsolete in current OpenROAD. The tool prints a warning and then skips pin placement entirely. With no pin locations, global placement gave up with "clk toplevel port is not placed", detailed placement rejected the clock terminal, and the timing engine quietly fell back to reporting the unplaced estimate.

The fix. Remove the dead option. The more useful fix was making the flow stop when a step fails, which is next.

6.3 Errors that never propagated

The bug above could hide because a failing step produced plausible output instead of stopping. OpenROAD's Python API returns command results as a string and never raises, so a Tcl error, a misspelled option, a missing file and a successful run all look the same to the calling script.

Now every command is wrapped so a non-zero return becomes a Python exception that stops the flow and prints the failing command. That one change immediately surfaced three more problems that had been silent: the pin placement failure, an infinite evaluation loop in my timing constraints caused by `remove_from_collection`, and a wrong argument form for `repair_tie_fanout`, which wants a cell and port separated by a slash. A flow that fails loudly is worth more than a flow that produces numbers.

6.4 A test that only failed when other tests ran first

What happened. A hazard test failed at retired instruction 224 in the full regression and passed on its own.

Why. The register file has no reset, which is correct for real hardware: resetting 992 flip-flops costs area and power for nothing, and software is supposed to write a register before reading it. In simulation that means registers keep their values

from the previous program while the reference model starts at zero.

The fix. The testbench clears the architectural state between programs. This is a testbench bug, not a design bug, and the distinction matters: "fixing" it by adding a reset to the register file would have put real silicon cost into the chip to paper over a problem in my test harness.

6.5 A simulator that crashed on a repeated argument

Icarus Verilog 12.0 segfaults during elaboration if you name the top module twice with `-s`. cocotb's makefile already passes it, so my extra copy crashed every build. Removing the duplicate fixed it. That one is a bug in the simulator's argument handling rather than in my design, but it cost me an afternoon.

7. Physical verification

Signoff asks a different question from the implementation tools. Not "did my router think it followed the rules" but "does the finished layout follow the foundry's rules". Running both is the entire point, and in this project the two disagreed.

7.1 Design rule checking

I streamed the routed design out to GDSII and checked it against the SkyWater rule deck in KLayout. OpenROAD's router reported zero violations of its own rule set. The signoff deck found 64, all the same kind.

Check	Violations	Rule	What it means
Router's own check	0	-	routing finished with no internal violations
Signoff deck, as routed	64	m3.6	met3 shape under the 0.24 µm² minimum area
Signoff deck, after patching	0	-	clean

Table 15. This disagreement is the reason independent signoff exists.

Every one of them is an isolated met3 landing pad at a via stack, 0.585 × 0.330 µm, which is 0.193 µm² against a 0.24 µm² minimum. This is not a missing rule definition either: the technology LEF does declare the area rule for met3, so the router had the information and did not act on it for these pads.

I wrote a repair step to fix them. For each undersized shape it grows the polygon along the layer's preferred direction until it meets the area rule, but only after checking that the grown shape still keeps the full 0.3 µm spacing to every other met3 shape, so it cannot create a short or a spacing violation while fixing an area one. All 64 were patched and the layout is now clean against the whole deck.

Writing that step produced three mistakes of its own, and every one was caught by re-running the rule deck rather than by reading my own code. The first two turned up while I was developing against the earlier 14.5 ns layout, which had 59 of these violations. The third showed up the first time I ran it on the final design and its 64.

- **Wrong layer.** I operated on GDS layer 68/20 thinking it was met3. It is met1. That left all 59 met3 violations untouched and added 7,469 new ones on met1, for 7,528 in total. met3 is 70/20.
- **Off the manufacturing grid.** I grew each shape symmetrically about its centre. Splitting an odd amount in half puts both edges off the 5 nm grid, which cleared nearly all the area violations and produced 232 off-grid errors in exchange.
- **Incomplete neighbour check.** I compared each candidate against the other met3 shapes but left out the remaining undersized ones, so a patch could grow into a neighbour that was itself still waiting to be patched. That made one spacing violation. Widening the spacing margin to 0.4 µm did not help: the violation stayed and the script simply refused to patch 31 more shapes.

Growing on one side only, by whole grid steps, and checking against every other met3 shape including the ones not yet patched, satisfies all three at once: 64 of 64 patched, none skipped, zero violations. What I took from it is that a repair step needs the same verification discipline as the thing it repairs. Checking the baseline first is what told me the spacing violation was mine and not the router's, and that was worth more than any amount of reasoning about which rule might apply.

7.2 Layout versus schematic

LVS extracts a netlist from the layout geometry and compares it against the circuit the tools thought they were building, which catches shorts, opens and mis-wiring that DRC cannot see. I wrote the reference netlist out of the routed database in CDL form, 10,422 instances, and compared it against the extracted layout with the SkyWater LVS deck. It matches.

```
INFO : Congratulations! Netlists match.
```

Together with the clean DRC run, this says the layout is consistent both with the foundry rules and with the netlist I verified.

7.3 What this does and does not prove

The chain holds together in one direction now. The RTL matches an independent model over 144 programs and every bin of the coverage model. The properties that have to hold for all inputs are proved. The synthesized netlist reproduces the RTL cycle for cycle. The layout matches that netlist under LVS. The layout follows the design rules. Timing closes at three corners.

What it does not prove is that you could manufacture this. Timing uses three library corners but one parasitic extraction, so the interconnect RC is typical at every corner. 129 slew and 8 capacitance violations are still failing. The antenna check reports 12 nets I did not repair. There is no metal fill, no density check, and no I/O ring or seal ring, because this is a core macro and not a whole chip. I would rather list these than have someone else find them.

8. What I would do differently

8.1 Where the area goes

This design is register file. Mapping 992 bits of architectural state to individual flip-flops is most of the 1,375 flip-flops in the netlist, and flip-flops are the most expensive cells in the library by area. A real design would use a register file macro with two read ports and one write port, which would cut the area a lot and probably help timing too, by removing the wide multiplexers the flip-flop version needs.

8.2 Where the time goes, and this one is specific

The design runs at 69 MHz typical and 32.3 MHz across PVT, and the reason is not vague. The slow-corner critical path report shows a chain of **29 majority gates in series**. A majority gate is the carry output of a full adder, so that chain is a ripple-carry adder: each bit's carry feeds the next one, and the delay grows linearly with how wide the operands are. One structure is most of my cycle time.

The fix is well understood. A carry-lookahead or carry-select adder trades area for a logarithmic carry chain instead of a linear one. Yosys will infer a faster structure if the addition gets its own module and the mapper is allowed to use it, and ABC can be pushed harder with a tighter target. After the adder, the execute stage also does forwarding selection, branch comparison and target selection in the same cycle, so moving branch comparison into decode would be next. Quoting a frequency without naming the structure behind it would be backwards: the number is the consequence, the carry chain is the cause.

8.3 Everything that is still wrong with it

- **Not implemented:** CSRs, interrupts, compressed instructions, multiply and divide, caches, misaligned memory accesses, and any privilege mode beyond an implied machine mode.
- **Exceptions are non-standard.** A trapping instruction halts the core instead of jumping to a handler, because there is no trap vector and no CSR file. That was a deliberate scope decision and the reference model does exactly the same thing, but it is not what a specification-compliant machine does.
- **Two compliance tests are skipped.** `fence_i` needs the Zifencei extension for self-modifying code and `ma_data` needs misaligned access support. Both are outside what I implemented, and I would rather say so than quietly drop them.
- **Parasitics are single-corner.** Timing runs at three library corners, but the SPEF comes from one extraction, so the interconnect RC is typical everywhere. A full signoff also extracts minimum and maximum RC and crosses them with the library corners.

- **129 slew and 8 capacitance violations remain**, worst slew slack -0.10 ns. A lot of them are on the sleep pins of `lpflow_isobufsrc_1` cells. The design is DRC and LVS clean and meets setup and hold at every corner, but it is not clean on electrical checks, and those are different things.
- **Antenna violations were reported and not repaired** (12 nets), and there is no metal fill or density verification.
- **Synthesis picked 45 low-power isolation cells** that the reference flow excludes. My don't-use list is applied during place and route, which only constrains what the optimizer adds, not what synthesis already chose; it needs to go to ABC at technology mapping too. Those cells are where most of the slew violations live.
- **The memories are idealized.** Both return data in the same cycle, which no real memory does. Attaching an SRAM with one cycle of latency would need stall logic in fetch and memory that the core does not currently have.

8.4 If I kept going

In order: replace the ripple-carry adder, which is the single change that would most improve the frequency; exclude the low-power cell family at synthesis and rebuild, which should clear most of the slew violations; swap the flip-flop register file for a macro to cut area; add a minimal CSR file and trap vector so the exception behaviour is actually compliant; cross the library corners with min and max RC extraction; repair the antenna violations and add fill; and push the formal work from invariants to a full instruction-by-instruction correctness proof, which would make a lot of the simulation redundant.

9. Running it yourself

```
# build the compliance suite
./scripts/build_riscv_tests.sh

# simulation: compliance, directed and random, with coverage
make -C tb MODULE=test_core RANDOM_SEEDS=12

# the five formal proofs
cd formal && for t in alu decode regfile regfile_persist core; do sby -f $t.sby; done

# synthesis
LIB_FILE=... OUT_DIR=synth/out CLK_PERIOD_PS=31000 yosys -c synth/synth.tcl

# the synthesized netlist against the cell models
make -C tb -f Makefile.gls SIM_BUILD=sim_build_gls

# place and route, extraction, timing
openroad -python pnr/flow.py pnr/config_mc.json

# three-corner timing on the finished design
openroad -python signoff/multicorner_sta.py pnr/config_mc.json

# GDS, min-area patch, DRC and LVS
klayout -b -r scripts/def2gds.py
klayout -b -r signoff/minarea_patch.py
klayout -b -rd in_gds=... -r .../drc/sky130hd.lydrc
klayout -b -rd in_gds=... -rd cdl_file=... -r .../lvs/sky130hd.lylvs
```

HANDOFF.md in the project bundle has the full setup, including the Docker route for machines where the OpenROAD wheel will not run natively, and what each stage should print if it worked.

Appendix. Where everything lives

	Path	Contents
RTL	rtl/*.v	the four synthesizable modules

	Path	Contents
Reference model	model/iss.py	the independent RISC-V simulator
Test generators	model/gen.py	directed and random programs
Testbench	tb/test_core.py	the instruction-by-instruction checker and coverage model
Netlist testbench	tb/test_gls.py	gate-level regression
Formal properties	formal/props.sv	the five suites
Synthesis	synth/synth.tcl	the Yosys script
Place and route	pnr/flow.py	floorplan through extraction and timing
Repair step	signoff/minarea_patch.py	the minimum-area fix
Layout	signoff/rv32i_core_mc_drc_lvs_clean.gds	the finished GDSII
Results	results/*.json	coverage, simulation, sweep, summary
Tool reports	pnr/run_*/ signoff/*.lyrdb	timing, power, DRC and LVS